谢reed

📞 (+86) 18518900987 | 📧 2132033360@qq.com | 🌐 Nanjing, China

🎯 求职意向：Python 后端开发工程师

🧢 个人简介s

3 年 Python 后端开发经验，熟悉 Django/Flask/FastAPI 开发全流程，具备独立完成功能模块设计、编码、测试与上线的能力。在电商、企业服务领域有多个项目实战经验，擅长**接口开发、数据库优化、异步任务处理与基础 DevOps 协作**。注重代码质量与可维护性，能高效协同产品、前端、测试推进项目交付。

💻 技术能力

类别	掌握程度与典型应用场景
语言与框架	Python 3.7+（熟练）、Django + DRF（主力）、Flask（熟悉）、FastAPI（项目实践）
Web 开发	RESTful API 设计、JWT/OAuth2 认证、中间件开发、异常统一处理、Swagger 文档集成
数据库	MySQL（索引优化/慢查询分析）、PostgreSQL（基础）、Redis（缓存/分布式锁/队列）
异步与消息	Celery + Redis/RabbitMQ（定时/异步任务）、requests 异步调用（aiohttp 基础）

运维与工具	Git（分支管理）、Docker（镜像构建/容器调试）、Linux（日志排查/基础命令）、Nginx（配置代理/静态资源）
测试与质量	pytest（单元/接口测试）、Postman（接口调试）、Sentry（错误监控）、基础 CI/CD（GitLab Runner）
协作能力	熟练使用 Jira/Confluence/飞书，可独立对接需求、评估工时、撰写技术方案文档

🧬 项目经历（真实中级定位 · 可落地 · 可深挖）

🛒 B2B 电商平台后台系统（Django + DRF）｜核心后端开发｜2023.02 – 2024.11

🔹 *背景：为制造业客户提供商品管理、订单处理、客户 CRM 一体化后台，日均订单 5K+*

🔹 *团队：5 人（2 后端+2 前端+1 测试），我负责订单&客户模块*

🔧 主要职责与实现：

- **订单管理模块**
  - 独立开发订单 CRUD、状态流转（待支付→已发货→已完成）、退款申请接口（DRF ModelViewSet）

    - 优化订单列表查询：
        - 添加复合索引（status + created_at）
        - 分页改用 cursor pagination（避免 deep offset）
        - 查询耗时从 1.2s → 220ms（10 万+数据）
    - 状态变更通知：通过 Celery 异步发送邮件/站内信（避免阻塞主流程）
- **客户分层与标签系统**
    - 设计客户标签模型（多对多），支持手动打标 + 规则自动打标（如：近 30 天采购≥5 次 → “高活跃”）
    - 实现标签筛选 API：支持组合条件（AND/OR），使用 Q 对象动态构建查询
    - 缓存优化：高频标签列表缓存至 Redis（TTL=5min），减少 DB 压力 30%
- **协作与交付**
    - 与前端约定接口规范（JSON Schema），使用 drf-yasg 生成 Swagger 文档
    - 编写单元测试（pytest + factory_boy），核心接口覆盖率 ≥ 75%
    - 参与 Code Review，推动团队统一异常处理格式（统一返回 {code, msg, data}）

✅ **成果：**

- 模块按期交付 6 个迭代版本，0 线上 P0 级缺陷
- 订单导出功能优化：分页导出 + 异步生成 Excel（Celery + openpyxl），支持 10 万行数据
- 获团队“季度交付之星”

企业微信 SCRM 系统（Flask + Vue）| 后端主力开发 | 2021.09 – 2023.01

🔹 *目标：集成企业微信 API，实现客户线索管理、SOP 任务提醒、数据看板*

🔧 **关键工作：**

- **企业微信对接**
  - 封装企业微信 API SDK（客户联系、消息发送、部门同步），支持 access_token 自动刷新
  - 实现客户添加自动打标：员工扫码添加客户 → 触发 webhook → 更新客户来源标签
- **SOP（标准作业流程）任务引擎**
  - 设计任务模型：触发条件（时间/事件） + 执行动作（发消息/提醒员工）
  - 定时任务：Celery Beat 每 5 分钟扫描待执行任务，支持动态启停
  - 避免重复执行：Redis 分布式锁（redis-py + set nx ex）
- **数据看板后端**
  - 提供聚合接口：今日新增客户、7 日转化率、员工跟进统计等
  - 使用 sqlalchemy.func 实现数据库层聚合（非 Python 循环），提升响应速度
  - 缓存策略：看板首页数据缓存 2 分钟（flask-caching）